

School of Electrical and Computer Engineering		Spring 1404
Reinforcement Learning		Final Project

Post Delivery Robot with Deep Q-Learning (DQN)

In this project, you will simulate a post-delivery robot that must pick up a package at one location within a gridworld environment (as in CA3 and CA4) and then deliver it to a destination location. **The episode ends when the package is successfully dropped off at the correct destination or robot moves a maximum number of steps.**

The environment is a 5×5 grid.

The robot can move: **North, South, East, West**, and also **Pick Up/Drop-off** packages.

Rewards:

- Successful Drop-off: **+20**
- Illegal move*: **-10**
- Every move: **-1**

* Illegal moves:

- Pick-up when **no package** is at the robot's location
- Drop-off when:
 - The robot is **not carrying** the package
 - The robot is **not at the correct** destination
- Moving **into a wall**

At the start of each episode:

- The package is randomly positioned at one **of the four marked cells** (Figure 1)
- The destination of the package randomly selected from the **same four marked cells**
- The robot starts from a **random location** on the grid

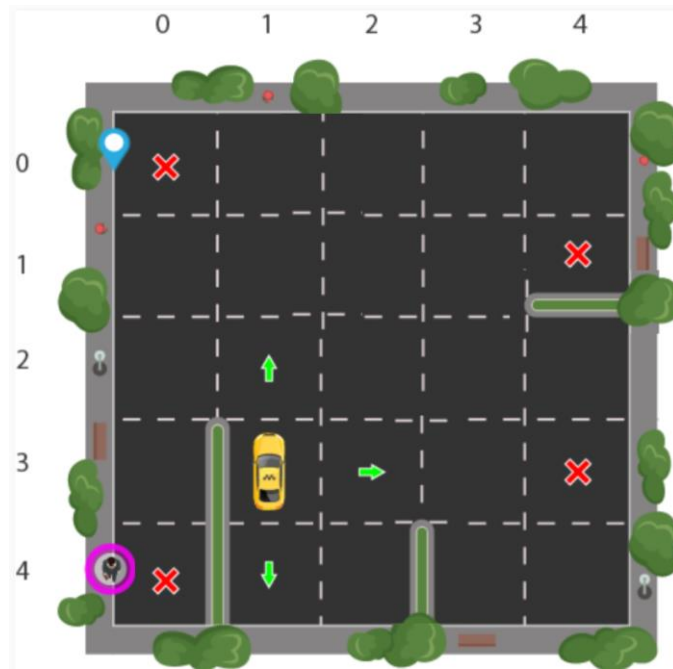


Figure 1: A 5×5 gridworld. The four marked cells indicate possible pickup locations for the package.

Part 1: Environment Setup

Model the environment as a 5×5 **gridworld**.

Define the action space: **North, South, East, West, Pick Up** and **Drop-off**

Define the state space:

- Robot's position: $5 \times 5 = 25 \rightarrow 25$ values
- Package location: One of the 4 locations or "with robot" $\rightarrow 5$ values
- Destination location: One of the 4 locations $\rightarrow 4$ values

Thus, the total number of unique states is:

$$25(\text{robot positions}) \times 5(\text{package states}) \times 4(\text{destinations}) = 500$$

Initialize the environment at the start of each episode:

- A **package** is placed at a randomly selected **marked cell**
- The **destination** is randomly selected from the **marked cells**

Episode Termination: (it is related to *done* attribute in replay buffer of DQN)

- When the package is **successfully dropped off**
- Or after "**Max steps per episode**" steps.

Note: The robot is aware of the **package's location** and **destination** at **the start of the episode**.

Handle rewards and penalties according to the problem description.

Part 2: Solve Using Deep Q-Network (DQN)

To solve this extended post delivery problem, you must implement a Deep Q-Network (DQN), a reinforcement learning algorithm that combines Q-learning with function approximation using a neural network.

DQN is used here because the state space is too large for traditional Q-tables (500 discrete states), and a neural network can generalize better across unseen or rarely visited states.

Your DQN agent must use a simple fully connected feedforward neural network to approximate the Q-value function $Q(s, a)$. The network should consist of:

- **Input:** A single integer representing the environment state (0–499), which should be passed through an embedding layer to produce a dense vector representation.
- **Hidden Layer:** A single hidden layer with 128 ReLU units
- **Output Layer:** A layer with 6 units, one for each possible action

This means the network maps each state to a vector of 6 Q-values.

DQN Components:

- Experience Replay Buffer
 - Store transitions in the format: *(state, action, reward, next_state, done)*
 - Sample random mini-batches from this buffer to update the Q-network
 - Helps decorrelate the training data and stabilize learning
- Target Network
 - Use a second, frozen Q-network (the target network) to compute the target Q-values
 - Periodically copy weights from the main Q-network to the target network (e.g., every 100 steps)
 - Prevents harmful feedback loops during learning

Recommended setting:

Epsilon-Greedy Strategy:

- Start with $\varepsilon = 1.0$, decay gradually to $\varepsilon = 0.01$

Hyperparameters:

Learning rate (α): 0.0005	Discount factor (γ): 0.99	Replay buffer size: 50,000
Batch size: 32	Max steps per episode: 100	Target Update Frequency: Every 100 steps
Number of episodes for training > 10,000		

Some Useful links for DQN:

<https://medium.com/@samina.amin/deep-q-learning-dqn-71c109586bae>

https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html

Required Outputs:

1. Plot of **average return** per episode (e.g., smoothed over 100-episode windows)
 2. Visualization of learned policy
 3. Loss curve over training (optional)
-

Submission Requirements:

- The **source code** for the environment, DQN agent and the training script.
 - Any plots you generate.
 - A **short report** explaining your approach (Network architecture, Hyperparameters used, etc.) and findings, and instructions on how to run your code.
-